



LẬP TRÌNH GAME 3D NÂNG CAO

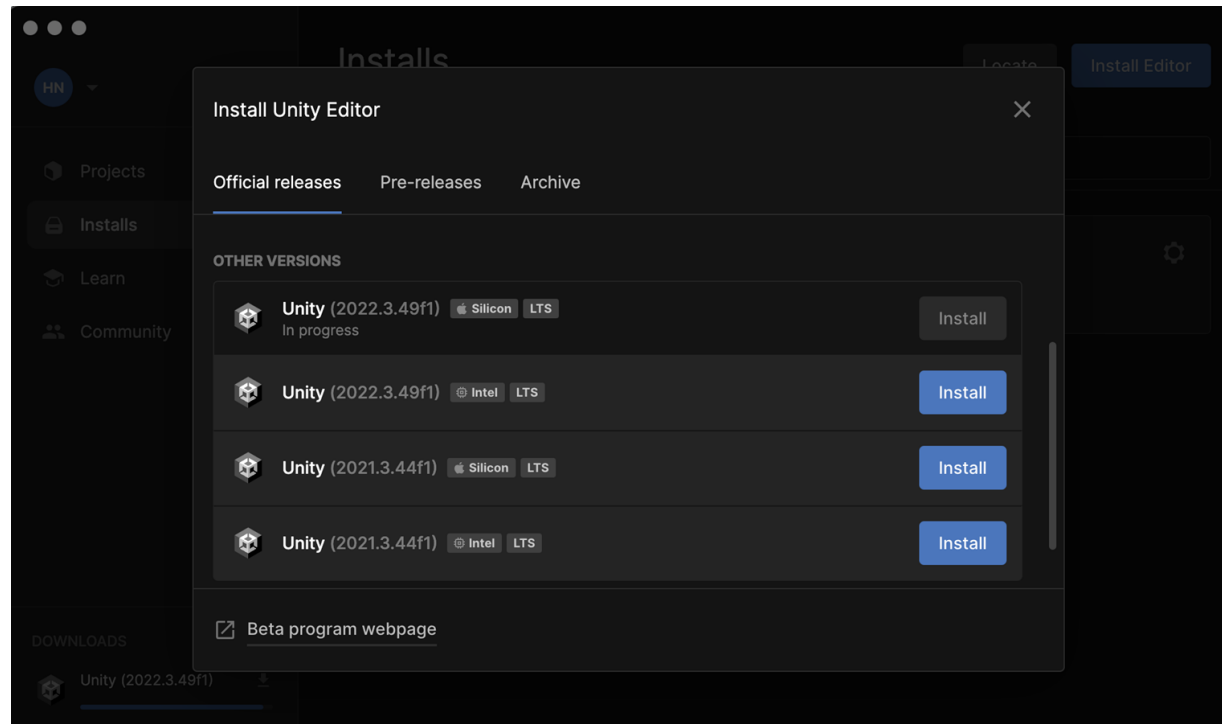
BÀI 1 : COROUTINES TRONG UNITY 3D

- ☐ Ôn tập Game 3D cơ bản
- ☐ Coroutines là gì ?
- ☐ Các trường hợp sử dụng Coroutines
- ☐ Ví dụ

- ❑ Phần mềm Unity3D - Unity Editor
- ❑ Cập nhật phiên bản Editor mới nhất, phù hợp với thiết bị
- ❑ Sử dụng [RIDER IDE](#) để lập trình



- ☐ Lựa chọn phiên bản Intel hoặc Silicon phù hợp với máy tính - laptop
- ☐ **Không sử dụng phiên bản Preview**



- ☐ Các khái niệm cần biết trong Unity 3D
- ☐ Khái niệm GameObject ,
- ☐ Component : Transform, Rigidbody, Collider, Script.

- ❑ **Coroutines** là một cách để thực hiện một thao tác theo thời gian thay vì ngay lập tức
- ❑ “Một thao tác theo thời gian” có nghĩa là thực hiện một nhiệm vụ hoặc hành động không xảy ra ngay lập tức, mà diễn ra từ từ hoặc theo từng giai đoạn trong một khoảng thời gian nhất định

❑ Sau x thời gian thì pháo sẽ bắn 1 viên đạn



- ❑ VD : Nhân vật di chuyển từ vị trí này đến vị trí khác trong một khoảng thời gian nhất định, thay vì di chuyển ngay lập tức
- ❑ Coroutines còn được sử dụng trong trường hợp xử lý các sự kiện hoặc chức năng riêng biệt mà không ảnh hưởng tới Main Thread (Luồng chính) của game, **không gây gián đoạn game**

- ❑ Coroutines không giới hạn số lượng sử dụng trong Unity
- ❑ Chạy quá nhiều coroutines cùng một lúc có thể tiêu tốn tài nguyên hệ thống gây lag, giật do được lưu trên bộ nhớ **heap**
- ❑ Số lượng Coroutines nhiều khiến game trở nên phức tạp, khó kiểm soát

□ Coroutines được khởi tạo bởi kiểu dữ liệu

IEnumerator và hàm **StartCoroutine**

□ Xem xét lần lượt các ví dụ thực tế sử dụng

Coroutines trong Unity 3D

❑ Xuất hiện GameObject sau 3s

```
13      public GameObject enemyPrefab;  ✎ Unchanged
14
15      ✎ Event function
16      private void Start()
17      {
18          StartCoroutine(routine:WaitAndSpawnEnemy());
19      }
20      ✎ Frequently called  ✎ 1 usage
21      IEnumerator WaitAndSpawnEnemy()
22      {
23          yield return new WaitForSeconds(3f); // Chờ 3 giây
24          Instantiate(enemyPrefab, Vector3.zero, Quaternion.identity); // Tạo ra đối tượng kẻ thù
25      }
```

❑ Xuất hiện liên tục cách nhau 3s

```
13      public GameObject enemyPrefab;  Cube
14
15       Event function
16      private void Start()
17      {
18          StartCoroutine(routine: WaitAndSpawnEnemy());
19      }
20
21       Frequently called  1 usage
22      IEnumerator WaitAndSpawnEnemy()
23      {
24          while (true) // bổ sung vòng lặp while
25          {
26              yield return new WaitForSeconds(3f); // Chờ 3 giây
27              Instantiate(enemyPrefab, Vector3.zero, Quaternion.identity); // Tạo ra đối tượng kẻ thù
28          }
29      }
```

❑ Di chuyển mượt mà GameObject từ vị trí A -> B

```
14     private void Start()
15     {
16         // di chuyển gameObject từ vị trí hiện tại tới vị trí (0,0,0)
17         StartCoroutine(routine: MoveObject(transform, Vector3.zero, duration: 5f));
18     }
19     IEnumerator MoveObject(Transform obj, Vector3 target, float duration)
20     {
21         Vector3 startPosition = obj.position;
22         float elapsedTime = 0f;
23
24         while (elapsedTime < duration)
25         {
26             // Di chuyển từ từ từ A đến B
27             obj.position = Vector3.Lerp(a: startPosition, b: target, t: elapsedTime / duration);
28             elapsedTime += Time.deltaTime;
29             yield return null; // Chờ đến frame tiếp theo
30         }
31
32         obj.position = target; // Đảm bảo đối tượng đến đúng vị trí cuối cùng
33     }
```

□ Thu nhỏ hoặc phóng to GameObject khi ăn vật phẩm

```
14 private void Start()
15 {
16     // di chuyển gameObject từ vị trí hiện tại tới vị trí (0,0,0)
17     StartCoroutine(routine: ScaleOverTime(transform, targetScale: new Vector3(x:3, y:3, z:3), duration:5f));
18 }
19
20 // Frequently called ? usages ? overrides ? extension methods ? exposing APIs
21 IEnumerator ScaleOverTime(Transform obj, Vector3 targetScale, float duration)
22 {
23     Vector3 startScale = obj.localScale;
24     float elapsedTime = 0f;
25
26     while (elapsedTime < duration)
27     {
28         obj.localScale = Vector3.Lerp(a:startScale, b:targetScale, t:elapsedTime / duration); // Thu/phóng đối tượng
29         elapsedTime += Time.deltaTime;
30         yield return null;
31     }
32
33     obj.localScale = targetScale; // Đảm bảo đối tượng đạt kích thước mục tiêu
34 }
```



Giảng viên giao bài tập



LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 1: WAIT TIME TRONG UNITY 3D

- ❑ Wait time : 1 khái niệm về các câu lệnh cho phép xử lý bất đồng bộ trong game Unity
- ❑ Tạo các lệnh chờ xử lý các nhiệm vụ trong trò chơi 1 cách mượt mà
- ❑ Unity cung cấp 6 câu lệnh wait time khác nhau

- ☐ WaitForEndOfFrame
- ☐ WaitForFixedUpdate
- ☐ WaitForSeconds
- ☐ WaitForSecondsRealtime
- ☐ WaitUntil
- ☐ WaitWhile

- ❑ WaitForEndOfFrame : Lệnh này sẽ chờ cho đến khi khung hình hiện tại kết thúc, sau đó mới tiếp tục thực thi đoạn mã tiếp theo.
- ❑ Ví dụ : Thực hiện chụp ảnh lại màn chơi Game sau khi khung hình kết thúc để đảm bảo ảnh chụp được đầy đủ

```

19  IEnumerator CaptureScreen()
20  {
21      // Chờ khi kết thúc Frame, các đối tượng đã được vẽ
22      yield return new WaitForEndOfFrame();
23      // Lấy kích thước của màn hình
24      int width = Screen.width;
25      int height = Screen.height;
26      Texture2D tex = new Texture2D(width, height, TextureFormat.RGB24, mipChain: false);
27      // Nạp thông tin pixel
28      tex.ReadPixels(source: new Rect(x: 0, y: 0, width, height), destX: 0, destY: 0);
29      tex.Apply();
30      // mã hóa dữ liệu dạng PNG
31      byte[] bytes = tex.EncodeToPNG();
32      Destroy(tex);
33      // Lưu file vào folder
34      string path = Application.dataPath + "/../SavedScreen.png";
35      Debug.Log(message: "Saving screenshot to: " + path);
36      Directory.CreateDirectory(Path.GetDirectoryName(path));
37      File.WriteAllBytes(path, bytes);
38  }

```

- ❑ **WaitForSeconds** : Lệnh này chờ trong một khoảng thời gian nhất định dựa trên thời gian trò chơi (`Time.timeScale`), có thể bị ảnh hưởng bởi tốc độ thời gian trong game.
- ❑ Ví dụ : Tạo các sự kiện đếm ngược, thời gian hồi chiêu (cooldown) hoặc tạm dừng hành động trong trò chơi.

```
? usages  ? overrides  ? extension methods  ? exposing APIs
20  IEnumerator ExampleCoroutine()
21  {
22      // In ra thời gian khi hàm này được gọi lần đầu tiên
23      Debug.Log(message: "Bắt đầu Coroutine tại thời điểm: " + Time.time);
24      // Tạm dừng Coroutine và chờ trong 5 giây
25      yield return new WaitForSeconds(5);
26      // Sau khi chờ 5 giây, in ra thời gian một lần nữa
27      Debug.Log(message: "Kết thúc Coroutine tại thời điểm: " + Time.time);
28  }
```

- ❑ **WaitForSecondsRealtime** : Lệnh này chờ trong một khoảng thời gian nhất định dựa trên **thời gian thực**
- ❑ Ví dụ : Thực hiện 1 lệnh tính theo thời gian thực tế mà không phụ thuộc vào game hay bị dừng lại khi game tạm dừng

```
private void Start()
{
    StartCoroutine(routine: WaitFewSeconds());
}

Frequently called ? usages ? overrides ? extension methods ? exposing APIs More...
IEnumerator WaitFewSeconds()
{
    print(Time.time);
    yield return new WaitForSecondsRealtime(time: 5);
    print(Time.time);
}
```


- ❑ **WaitUntil** : Lệnh này chờ cho đến khi một điều kiện cụ thể trở thành true.
- ❑ Ví dụ: Dừng một hành động cho đến khi một sự kiện hoặc điều kiện nào đó được thỏa mãn, chẳng hạn như đợi cho đến khi người chơi thu thập đủ tài nguyên.
- ❑ Hiện thị Game Over khi nhân vật hết máu

```

16      public int item;  ✖ Unchanged
      ✖ Event function  ? usages  ? overrides  ? extension methods  ? exposing APIs
17      void Start()
18      {
19          StartCoroutine(routine: Example());
20      }
      ✖ Frequently called  ? usages  ? overrides  ? extension methods  ? exposing APIs
21      IEnumerator Example()
22      {
23          Debug.Log(message: "Chờ đến khi thu thập đủ 10 items");
24          yield return new WaitUntil(() => item >= 10);
25          Debug.Log(message: "Princess was rescued!");
26      }
      ✖ Event function  ? usages  ? overrides  ? extension methods  ? exposing APIs
27      void Update()
28      {
29          if (item <= 10)
30          {
31              Debug.Log(message: "Frame: " + item);
32              item++;
33          }
34      }
  
```

- ❑ **WaitWhile** : Lệnh này sẽ tiếp tục chờ trong khi một điều kiện còn là true.
- ❑ Ví dụ: Dừng hành động trong khi người chơi nhấn giữ một nút hoặc trong khi một điều kiện nào đó còn đúng.

```

public int frame; ❸ Unchanged
❸ Event function ? usages ? overrides ? extension methods ? exposing APIs
void Start()
{
    StartCoroutine(routine: Example());
}
❸ Frequently called ? usages ? overrides ? extension methods ? exposing APIs
IEnumerator Example()
{
    Debug.Log(message: "Công chúa đứng im chờ hoàng tử cứu");
    yield return new WaitForSeconds(10);
    Debug.Log(message: "Công chúa thoát nên cuối cùng hoàng tử đã tới");
}
❸ Event function ? usages ? overrides ? extension methods ? exposing APIs
void Update()
{
    if (frame <= 10)
    {
        Debug.Log(message: "Frame: " + frame);
        frame++;
    }
}

```

- ❑ **WaitForFixedUpdate** : Lệnh này sẽ chờ cho đến khi hàm FixedUpdate kế tiếp được gọi
- ❑ Ví dụ: Đồng bộ các hành động dựa trên các tính toán vật lý của game

```
public float speed = 5f; ✖ Unchanged

✖ Event function ? usages ? overrides ? extension methods ? exposing APIs

void Start()
{
    // Bắt đầu coroutine để di chuyển đối tượng sau mỗi lần cập nhật vật lý
    StartCoroutine(routine: MoveAfterPhysics());
}

// Sử dụng Update để di chuyển đối tượng theo tốc độ khung hình
✖ Event function ? usages ? overrides ? extension methods ? exposing APIs

void Update()
{
    // Di chuyển đối tượng sang phải theo tốc độ khung hình
    transform.position += Vector3.right * speed * Time.deltaTime;
}
```

```
public float speed = 5f; ✖ Unchanged

✖ Event function ? usages ? overrides ? extension methods ? exposing APIs

void Start()
{
    // Bắt đầu coroutine để di chuyển đối tượng sau mỗi lần cập nhật vật lý
    StartCoroutine(routine: MoveAfterPhysics());
}

// Sử dụng Update để di chuyển đối tượng theo tốc độ khung hình
✖ Event function ? usages ? overrides ? extension methods ? exposing APIs

void Update()
{
    // Di chuyển đối tượng sang phải theo tốc độ khung hình
    transform.position += Vector3.right * speed * Time.deltaTime;
}
```



Giảng viên giao bài tập



Kết thúc